

BayesianDEB: Bayesian Dynamic Energy Budget Modelling in R

Package Manual and Tutorial (Version 0.1.0)

Branimir K. Hackenberger

April 2026

Abstract

BayesianDEB is an R package that provides a complete Bayesian framework for Dynamic Energy Budget (DEB) modelling. It wraps pre-written **Stan** models with a declarative R interface for data preparation, prior specification, MCMC fitting via **cmdstanr**, convergence diagnostics, posterior predictive checks, derived quantity estimation, and publication-quality visualisation. Four model types are included: individual-level growth, growth with reproduction, hierarchical multi-individual analysis with partial pooling, and DEBtox (toxicokinetic-toxicodynamic) models for ecotoxicology. This document provides a comprehensive reference for all exported functions, bundled Stan models, example datasets, and a step-by-step tutorial covering the full *Bayesian DEB* workflow.

Contents

1	Introduction	2
2	Installation	2
3	DEB Theory Recap	2
4	Package Architecture	3
4.1	Bundled Stan Models	3
5	Complete Function Reference	4
5.1	Data Preparation	4
5.1.1	bdeb_data() — Prepare Data for BDEB Models	4
5.1.2	repro_to_intervals() — Convert Cumulative Reproduction to Intervals	4
5.2	Prior Specification	5
5.2.1	Prior Constructors	5
5.2.2	prior_default() — Weakly Informative Defaults	5
5.3	Model Specification	6
5.3.1	bdeb_model() — Specify a BDEB Model	6
5.3.2	Observation Model Constructors	6
5.4	Model Fitting	6
5.4.1	bdeb_fit() — Fit a BDEB Model	6
5.5	Diagnostics and Posterior Analysis	7
5.5.1	bdeb_diagnose() — MCMC Convergence Diagnostics	7
5.5.2	bdeb_summary() — Posterior Summary Table	7
5.5.3	bdeb_derived() — Derived Biological Quantities	7
5.5.4	bdeb_ppc() — Posterior Predictive Checks	7

5.5.5	<code>bdeb_predict()</code> — Posterior Predictions	8
5.6	Plotting	8
5.6.1	<code>plot(fit, type = ...)</code> — Plot a BDEB Fit	8
5.6.2	<code>plot(ppc)</code> — Plot Posterior Predictive Checks	8
5.7	DEBtox (Ecotoxicology)	8
5.7.1	<code>bdeb_tox()</code> — DEBtox Model Specification	8
5.7.2	<code>bdeb_ec50()</code> — Extract EC50 and NEC	8
5.7.3	<code>plot_dose_response()</code> — Dose-Response Curve	9
5.8	Utility Functions	9
5.8.1	<code>arrhenius()</code> — Temperature Correction	9
5.8.2	<code>deb_fluxes()</code> — Compute DEB Energy Fluxes	9
6	Bundled Datasets	9
6.1	<code>eisenia_growth</code>	9
6.2	<code>folsomia_repro</code>	10
6.3	<code>debtox_growth</code>	10
7	Tutorial: Complete Workflow	10
7.1	Tutorial 1: Individual Growth	10
7.2	Tutorial 2: Hierarchical Multi-Individual	11
7.3	Tutorial 3: Growth + Reproduction	11
7.4	Tutorial 4: DEBtox Ecotoxicology	12
8	Stan Model Details	13
8.1	DEB ODE Right-Hand Side	13
8.2	Prior Hyperparameters as Data	13
8.3	Hierarchical Non-Centred Parameterisation	13
8.4	DEBtox: Damage Dynamics	14
9	Practical Advice	14
9.1	Choosing <code>adapt_delta</code>	14
9.2	When to Use Hierarchical Models	14
9.3	Prior Sensitivity	14
9.4	Computational Tips	14
10	Comparison with Existing Tools	14

1 Introduction

Dynamic Energy Budget (DEB) theory [Kooijman, 2010] provides a mechanistic, thermodynamically consistent description of how organisms acquire and utilise energy for maintenance, growth, development, and reproduction. Classical DEB calibration relies on deterministic optimisation—finding a single best-fit parameter vector—which yields no formal uncertainty quantification and cannot naturally accommodate prior biological knowledge or hierarchical data structures.

BayesianDEB addresses these limitations by embedding the DEB differential equation system within a Bayesian state-space framework. Parameters are treated as random variables with prior distributions; the likelihood is constructed from observation models linking latent DEB states to measured quantities; and the posterior is explored via Hamiltonian Monte Carlo (HMC) using Stan [Carpenter et al., 2017].

The package workflow follows the iterative Bayesian modelling cycle advocated by Gelman et al. [2020]:

1. Prepare and validate data.
2. Specify the DEB process model, priors, and observation model.
3. Fit the model via MCMC.
4. Diagnose convergence.
5. Perform posterior predictive checks.
6. Compute derived quantities and predictions.
7. Revise the model as needed.

2 Installation

BayesianDEB requires **cmdstanr** and a working CmdStan installation for model fitting. Data preparation, prior specification, and utility functions work without Stan.

```
# 1. Install cmdstanr from Stan r-universe
install.packages("cmdstanr",
  repos = c("https://stan-dev.r-universe.dev",
    getOption("repos")))

# 2. Install CmdStan (one-time, ~2 min)
cmdstanr::install_cmdstan()

# 3. Install BayesianDEB
install.packages("BayesianDEB")
# or from source:
# remotes::install_github("bhackenberger/BayesianDEB")

# 4. Load
library(BayesianDEB)
```

3 DEB Theory Recap

The standard DEB model tracks four state variables over time:

- E — reserve energy (J),
- V — structural volume (cm^3), with structural length $L = V^{1/3}$,
- E_H — maturity (J),
- E_R — reproduction buffer (J).

Energy flows are governed by the κ -rule: a fraction κ of mobilised reserve goes to somatic maintenance and growth; the remainder goes to maturity maintenance and reproduction. The

key fluxes are:

$$\dot{p}_A = f \{p_{Am}\} L^2 \quad (\text{assimilation}), \quad (1)$$

$$\dot{p}_C = \frac{E v L}{E + [E_G] V} \quad (\text{mobilisation}), \quad (2)$$

$$\dot{p}_M = [p_M] V \quad (\text{somatic maintenance}), \quad (3)$$

$$\frac{dE}{dt} = \dot{p}_A - \dot{p}_C, \quad (4)$$

$$\frac{dV}{dt} = \frac{\kappa \dot{p}_C - \dot{p}_M}{[E_G]}, \quad (5)$$

$$\frac{dE_R}{dt} = (1 - \kappa) \dot{p}_C - \dot{p}_J. \quad (6)$$

Table 1 lists the standard DEB parameters used throughout this package.

Table 1: Standard DEB parameters in **BayesianDEB**.

Symbol	R name	Units	Description
$\{p_{Am}\}$	p_Am	J d ⁻¹ cm ⁻²	Max. surface-area-specific assimilation
$[p_M]$	p_M	J d ⁻¹ cm ⁻³	Volume-specific somatic maintenance
κ	kappa	–	Allocation fraction to soma
v	v	cm d ⁻¹	Energy conductance
$[E_G]$	E_G	J cm ⁻³	Specific cost of structure
k_J	k_J	d ⁻¹	Maturity maintenance rate
E_H^p	E_Hp	J	Maturity at puberty
f	f_food	–	Scaled functional response (0–1)
E_0	E0	J cm ⁻³	Initial reserve density
L_0	L0	cm	Initial structural length
σ_L	sigma_L	cm	Observation error SD (length)

4 Package Architecture

BayesianDEB consists of three layers:

Stan models

Pre-written `.stan` files in `inst/stan/` implementing the DEB ODE system and Bayesian inference. Users never need to edit Stan code directly.

R interface

Exported functions for data preparation, model specification, fitting, diagnostics, and visualisation. All functions use S3 classes (`bdeb_data`, `bdeb_model`, `bdeb_fit`).

Prior system

A family of `prior_*`() constructor functions that encode distribution choices as lightweight objects passed to `bdeb_model`() .

4.1 Bundled Stan Models

All Stan models use the BDF (backward differentiation formula) stiff ODE solver (`ode_bdf`) with tolerances 10^{-6} and maximum 10 000 steps, appropriate for the time-scale separation typical of DEB reserve and structural dynamics.

Table 2: Stan models bundled with **BayesianDEB**.

Stan file	States	R type	Use case
bdeb_individual_growth	E, V	"individual"	Single organism growth
bdeb_growth_repro	E, V, R	"growth_repro"	Growth + reproduction
bdeb_hierarchical_growth	$E, V + RE$	"hierarchical"	Multi-individual, partial pooling
bdeb_debtox	E, V, R, D_w	"debtox"	Ecotoxicology (TKTD)

5 Complete Function Reference

5.1 Data Preparation

5.1.1 bdeb_data() — Prepare Data for BDEB Models

```
bdeb_data(growth = NULL, reproduction = NULL, survival = NULL,
          concentration = NULL, f_food = 1.0)
```

Converts long-format data frames into the structured format required by the Stan models. Validates column names, sorts by individual and time, and constructs the internal `bdeb_data` S3 object.

Arguments.

growth Data frame with columns `id`, `time`, `length`.
reproduction Data frame with columns `id`, `t_start`, `t_end`, `count`.
survival Data frame with columns `id`, `time`, `event` (1 = death, 0 = censored).
concentration Named numeric vector or data frame mapping IDs to external toxicant concentrations (for DEBtox).
f_food Scaled functional response, $f \in [0, 1]$. Default 1 (ad libitum).

Returns. A `bdeb_data` object (list with class "bdeb_data").

Example.

```
df <- data.frame(
  id = rep(1, 10),
  time = seq(0, 45, by = 5),
  length = c(0.10, 0.15, 0.22, 0.30, 0.38,
             0.44, 0.49, 0.52, 0.54, 0.55)
)
dat <- bdeb_data(growth = df)
dat
```

5.1.2 repro_to_intervals() — Convert Cumulative Reproduction to Intervals

```
repro_to_intervals(df)
```

Converts a data frame of cumulative offspring counts to interval counts suitable for the reproduction argument of `bdeb_data()`.

Arguments.

df Data frame with columns `id`, `time`, `cumulative`.

Returns. Data frame with columns `id`, `t_start`, `t_end`, `count`.

Example.

```

cumul <- data.frame(
  id = rep(1, 5),
  time = c(0, 7, 14, 21, 28),
  cumulative = c(0, 10, 30, 60, 100)
)
intervals <- repro_to_intervals(cumul)
intervals
#   id t_start t_end count
# 1  1      0     7    10
# 2  1      7    14    20
# 3  1     14    21    30
# 4  1     21    28    40

```

5.2 Prior Specification

Prior distributions are specified via lightweight constructor functions that return `bdeb_prior` objects. These are passed as a named list to `bdeb_model()`; any unspecified priors are filled from `prior_default()`.

5.2.1 Prior Constructors

Function	Distribution	Arguments
<code>prior_lognormal()</code>	$\text{LogNormal}(\mu, \sigma)$	<code>mu = 0, sigma = 1</code>
<code>prior_normal()</code>	$\mathcal{N}(\mu, \sigma)$	<code>mu = 0, sigma = 1</code>
<code>prior_beta()</code>	$\text{Beta}(a, b)$	<code>a = 2, b = 2</code>
<code>prior_halfnormal()</code>	$\text{HalfNormal}(\sigma)$	<code>sigma = 1</code>
<code>prior_halfcauchy()</code>	$\text{HalfCauchy}(\sigma)$	<code>sigma = 1</code>
<code>prior_exponential()</code>	$\text{Exponential}(\lambda)$	<code>rate = 1</code>

All return a `bdeb_prior` object (list with elements `family` and the hyperparameters).

Usage recommendations.

- Positive rate parameters ($\{p_{Am}\}, [p_M], v, [E_G]$): `prior_lognormal()`.
- Allocation fraction (κ): `prior_beta()`.
- Observation error SD (σ_L): `prior_halfnormal()`.
- Hierarchical variance components: `prior_exponential()`.

5.2.2 `prior_default()` — Weakly Informative Defaults

```

prior_default(type = c("individual", "growth_repro",
                      "hierarchical", "debtox"))

```

Returns a complete named list of weakly informative priors appropriate for the specified model type. Table 4 shows the defaults for the "individual" type.

Additional priors are included for specific types:

- "growth_repro" adds: `k_J`, `E_Hp`, `k_R`, `phi_R`.
- "hierarchical" adds: `mu_log_p_Am` (Normal), `sigma_log_p_Am` (Exponential).
- "debtox" adds: `k_d`, `z_w`, `b_w`.

Table 4: Default priors for individual growth model.

Parameter	Prior	Rationale
p_Am	LogNormal(1.0, 1.0)	Covers range ~ 0.4 –20
p_M	LogNormal(−1.0, 1.0)	Covers range ~ 0.04 –2
kappa	Beta(2, 2)	Favours interior values
v	LogNormal(−1.5, 1.0)	Covers range ~ 0.02 –1
E_G	LogNormal(6.0, 1.0)	Covers ~ 100 –3000
E0	LogNormal(0.0, 1.0)	
L0	LogNormal(−2.0, 1.0)	Initial length ~ 0.01 –0.5
sigma_L	HalfNormal(0.1)	Measurement error SD

5.3 Model Specification

5.3.1 bdeb_model() — Specify a BDEB Model

```
bdeb_model(data, type = c("individual", "growth_repro",
                          "hierarchical", "debttox"),
           priors = list(), observation = list(),
           temperature = NULL)
```

Creates a model specification combining data, DEB model type, observation model, and prior distributions. Does not fit the model.

Arguments.

data A bdeb_data object.
type Model type (see Table 2).
priors Named list of bdeb_prior objects. Missing entries filled from prior_default().
observation Named list: growth = obs_normal(), reproduction = obs_negbinom(), etc.
temperature Optional list with T, T_ref, T_A for Arrhenius correction.

Returns. A bdeb_model object.

5.3.2 Observation Model Constructors

Function	Distribution	Default for
obs_normal()	$\mathcal{N}(\hat{L}, \sigma_L)$	Growth (continuous)
obs_lognormal()	LogNormal($\log \hat{L}, \sigma$)	Multiplicative error
obs_student_t()	$t_\nu(\hat{L}, \sigma_L)$	Outlier-robust
obs_poisson()	Poisson(ΔR)	Count data
obs_negbinom()	NegBin($\Delta R, \phi$)	Reproduction (default)

5.4 Model Fitting

5.4.1 bdeb_fit() — Fit a BDEB Model

```
bdeb_fit(model, chains = 4, iter_warmup = 1000,
         iter_sampling = 1000, adapt_delta = 0.8,
         max_treedepth = 10, seed = NULL,
         parallel_chains = NULL, refresh = 200, ...)
```

Compiles the Stan model and runs MCMC sampling via **cmdstanr**.

Key arguments.

model A `bdeb_model` object.

adapt_delta Target acceptance probability. Increase toward 1.0 if divergences occur.

max_treedepth Maximum NUTS tree depth. Increase if treedepth is saturated.

seed Integer seed for reproducibility.

Returns. A `bdeb_fit` object containing the raw `CmdStanMCMC` result (`fit$fit`), the model specification (`fit$model`), and sampling metadata.

5.5 Diagnostics and Posterior Analysis

5.5.1 `bdeb_diagnose()` — MCMC Convergence Diagnostics

```
bdeb_diagnose(fit, pars = NULL)
```

Reports: divergent transitions, treedepth saturation, E-BFMI, $\hat{R}$, bulk and tail ESS for each parameter. Flags problematic values automatically.

Returns. Invisibly, a list with `n_divergent`, `n_max_treedepth`, `ebfmi`, and `summary` (a `posterior::draws_summary` data frame).

5.5.2 `bdeb_summary()` — Posterior Summary Table

```
bdeb_summary(fit, pars = NULL, prob = 0.90)
```

Returns a tidy summary of posterior draws: mean, SD, median, credible interval bounds, $\hat{R}$, and ESS.

Arguments.

pars Character vector of parameter names. Default: all.

prob Credible interval width. Default 0.90 (5th–95th percentiles).

5.5.3 `bdeb_derived()` — Derived Biological Quantities

```
bdeb_derived(fit, quantities = c("L_inf", "k_M",  
                                "growth_rate"), f = 1.0)
```

Computes derived quantities from posterior draws, propagating uncertainty:

- "L_inf": ultimate structural length, $L_\infty = f \kappa \{p_{Am}\} / [p_M]$.
- "k_M": somatic maintenance rate constant, $k_M = [p_M] / [E_G]$.
- "growth_rate": von Bertalanffy rate, $\dot{k} = \frac{v}{3} \frac{[p_M]}{\kappa [E_G]}$.

Returns. A `posterior::draws_df` with one column per requested quantity and one row per posterior draw.

5.5.4 `bdeb_ppc()` — Posterior Predictive Checks

```
bdeb_ppc(fit, type = c("growth", "reproduction", "all"))
```

Extracts replicated observations from the `generated quantities` block of the Stan model.

Returns. A `bdeb_ppc` object with matrices of replicated data (`L_rep`, `R_rep`) and the corresponding observed data.

5.5.5 `bdeb_predict()` — Posterior Predictions

```
bdeb_predict(fit, newdata = NULL, n_draws = 200)
```

Extracts posterior predicted trajectories. Can also be called as `predict(fit)`.

5.6 Plotting

5.6.1 `plot(fit, type = ...)` — Plot a BDEB Fit

```
plot(fit, type = c("trace", "posterior", "pairs",
                  "trajectory"),
     pars = NULL, n_draws = 100)
```

Available plot types:

"trace" MCMC trace plots for convergence assessment.

"posterior" Overlaid marginal posterior density curves.

"pairs" Bivariate scatter plots showing posterior correlations.

"trajectory" Posterior predicted growth curves overlaid on observed data.

All return `ggplot2` objects.

5.6.2 `plot(ppc)` — Plot Posterior Predictive Checks

```
ppc <- bdeb_ppc(fit)
plot(ppc, n_draws = 50)
```

Overlays replicated trajectories (grey) on observed data points (red).

5.7 DEBtox (Ecotoxicology)

5.7.1 `bdeb_tox()` — DEBtox Model Specification

```
bdeb_tox(data, stress = c("assimilation", "maintenance",
                          "growth_cost"), priors = list(), ...)
```

Convenience wrapper for `bdeb_model()` with `type = "debtox"`. Currently implements stress on assimilation (reduces $\dot{p}_A$ by a factor $\max(1 - s, 0)$ where $s = b_w \max(D_w, 0)$).

The internal damage D_w follows:

$$\frac{dD_w}{dt} = k_d(\max(C_w - z_w, 0) - D_w), \quad (7)$$

where k_d is the damage recovery rate, z_w is the no-effect concentration (NEC), and C_w is external concentration.

5.7.2 `bdeb_ec50()` — Extract EC50 and NEC

```
bdeb_ec50(fit, prob = 0.90)
```

Extracts the posterior distribution of the EC_{50} computed analytically in Stan as:

$$EC_{50} = z_w + \frac{0.5}{b_w}. \quad (8)$$

Returns. A list with `draws` (posterior vector), `summary` (data frame with mean, median, SD, credible interval), and `NEC` (summary of the no-effect concentration).

5.7.3 `plot_dose_response()` — Dose-Response Curve

```
plot_dose_response(fit, endpoint = "growth", n_draws = 100)
```

Plots the stress function $\max(1 - b_w \max(C - z_w, 0), 0)$ over a continuous concentration range with posterior uncertainty bands, overlaying observed endpoint values.

5.8 Utility Functions

5.8.1 `arrhenius()` — Temperature Correction

```
arrhenius(T, T_ref = 293.15, T_A = 8000)
```

Returns the Arrhenius correction factor:

$$c_T = \exp\left(\frac{T_A}{T_{\text{ref}}} - \frac{T_A}{T}\right). \quad (9)$$

Example.

```
# At reference temperature: factor = 1
arrhenius(293.15)
# [1] 1

# At 25C (5 degrees warmer): factor > 1
arrhenius(298.15)
# [1] 1.741
```

5.8.2 `deb_fluxes()` — Compute DEB Energy Fluxes

```
deb_fluxes(E, V, f, p_Am, p_M, kappa, v, E_G,
            k_J = 0, E_Hp = 0)
```

Given current state (E, V) and parameters, returns a named list of all standard DEB fluxes: `p_A`, `p_C`, `p_M`, `p_G`, `p_J`, `p_R`, plus structural length `L` and scaled reserve density `e`.

Example.

```
f1 <- deb_fluxes(E = 10, V = 0.5, f = 1.0,
                 p_Am = 5, p_M = 0.5, kappa = 0.75,
                 v = 0.2, E_G = 400)
f1$p_A # assimilation flux
f1$L   # structural length = V^(1/3)
```

6 Bundled Datasets

6.1 `eisenia_growth`

Simulated growth data for 21 individuals of the earthworm *Eisenia fetida* measured weekly over 12 weeks (days 0, 7, 14, ..., 84).

Rows 273 (21 individuals $\times$ 13 time points).

Columns `id` (integer 1–21), `time` (days), `length` (cm).

True parameters $\{p_{Am}\} = 5.0$, $[p_M] = 0.5$, $\kappa = 0.75$, $v = 0.2$, $[E_G] = 400$; individual variation in $\{p_{Am}\}$ (CV $\approx 10\%$) and L_0 (CV $\approx 5\%$); $\sigma_L = 0.015$ cm.

6.2 folsomia_repro

Simulated reproduction data for the springtail *Folsomia candida* exposed to 5 cadmium concentrations (0, 10, 50, 100, 500 mg/kg) with 6 replicates per concentration. Standard 28-day test.

Rows 30 (5 concentrations $\times$ 6 replicates).

Columns id, concentration, t_start, t_end, count.

True parameters NEC = 30 mg/kg, $b_w = 0.005$, base offspring = 120.

6.3 debtox_growth

Simulated growth under toxicant exposure: 4 concentration groups (0, 20, 80, 200), 10 individuals each, weekly measurements over 6 weeks.

Rows 280 (40 individuals $\times$ 7 time points).

Columns id, concentration, time, length.

True parameters NEC = 15, $b_w = 0.003$, stress on assimilation.

7 Tutorial: Complete Workflow

This section walks through all four model types using the bundled datasets.

7.1 Tutorial 1: Individual Growth

```
library(BayesianDEB)
data(eisenia_growth)

# --- Step 1: Prepare data (single individual) ---
df1 <- eisenia_growth[eisenia_growth$id == 1, ]
dat <- bdeb_data(growth = df1, f_food = 1.0)
dat

# --- Step 2: Specify model ---
mod <- bdeb_model(dat, type = "individual",
  priors = list(
    p_Am = prior_lognormal(mu = 1.5, sigma = 0.5),
    p_M = prior_lognormal(mu = -1.0, sigma = 0.5),
    kappa = prior_beta(a = 3, b = 2),
    sigma_L = prior_halfnormal(sigma = 0.05)
  ))
mod # prints specification including all priors

# --- Step 3: Fit via MCMC ---
fit <- bdeb_fit(mod, chains = 4, iter_sampling = 2000,
  adapt_delta = 0.9, seed = 42)
fit # prints quick diagnostic summary

# --- Step 4: Convergence diagnostics ---
bdeb_diagnose(fit)
# Check:
# - No divergent transitions
# - All R-hat < 1.01
# - Bulk ESS > 400 for all parameters

# Trace plots
plot(fit, type = "trace")
```

```

# Posterior correlations
plot(fit, type = "pairs",
     pars = c("p_Am", "p_M", "kappa"))

# --- Step 5: Posterior summary ---
bdeb_summary(fit, prob = 0.95)

# --- Step 6: Derived quantities ---
bdeb_derived(fit, quantities = c("L_inf", "growth_rate"))

# --- Step 7: Posterior predictive check ---
ppc <- bdeb_ppc(fit, type = "growth")
plot(ppc, n_draws = 100)

# --- Step 8: Trajectory plot ---
plot(fit, type = "trajectory", n_draws = 200)

```

7.2 Tutorial 2: Hierarchical Multi-Individual

```

data(eisenia_growth)

# All 21 individuals
dat_all <- bdeb_data(growth = eisenia_growth, f_food = 1.0)
dat_all
# => 21 individuals, 273 observations

# Hierarchical model: individual variation in p_Am
# Non-centred parameterisation (automatic in the Stan model)
mod_h <- bdeb_model(dat_all, type = "hierarchical",
  priors = list(
    mu_log_p_Am = prior_normal(mu = 1.5, sigma = 0.5),
    sigma_log_p_Am = prior_exponential(rate = 2),
    kappa = prior_beta(a = 3, b = 2)
  ))

fit_h <- bdeb_fit(mod_h, chains = 4, iter_sampling = 2000,
  adapt_delta = 0.95, seed = 123)

# Population-level summary
bdeb_summary(fit_h,
  pars = c("mu_log_p_Am", "sigma_log_p_Am",
    "p_M", "kappa", "v", "E_G"))

# Individual assimilation rates
bdeb_summary(fit_h,
  pars = paste0("p_Am_ind[", 1:5, "]"))

# Prediction for a new individual
bdeb_summary(fit_h, pars = "p_Am_new")

# Trace plots for hierarchical parameters
plot(fit_h, type = "trace",
  pars = c("mu_log_p_Am", "sigma_log_p_Am"))

```

7.3 Tutorial 3: Growth + Reproduction

```

# Simulated growth + reproduction data
growth_df <- data.frame(
  id = 1, time = seq(0, 56, by = 7),
  length = c(0.10, 0.18, 0.28, 0.37, 0.44,
             0.49, 0.52, 0.54, 0.55)
)

repro_df <- data.frame(
  id = 1,
  t_start = c(28, 35, 42, 49),
  t_end   = c(35, 42, 49, 56),
  count   = c(5, 12, 18, 15)
)

dat_gr <- bdeb_data(growth = growth_df,
                    reproduction = repro_df,
                    f_food = 1.0)

mod_gr <- bdeb_model(dat_gr, type = "growth_repro",
                    priors = list(
                      kappa = prior_beta(a = 3, b = 2),
                      k_R    = prior_lognormal(mu = -2, sigma = 1)
                    ))

fit_gr <- bdeb_fit(mod_gr, chains = 4,
                  adapt_delta = 0.95, seed = 99)

bdeb_diagnose(fit_gr)
bdeb_summary(fit_gr)

# Growth PPC
plot(bdeb_ppc(fit_gr, type = "growth"))

```

7.4 Tutorial 4: DEBtox Ecotoxicology

```

data(debttox_growth)

# Concentration mapping: group means
conc_map <- c(0, 20, 80, 200)
names(conc_map) <- unique(debttox_growth$concentration)

dat_tox <- bdeb_data(
  growth = debttox_growth,
  concentration = conc_map,
  f_food = 1.0
)

# DEBtox model: stress on assimilation
mod_tox <- bdeb_tox(dat_tox, stress = "assimilation",
                    priors = list(
                      z_w = prior_lognormal(mu = 2.5, sigma = 1.0),
                      b_w = prior_lognormal(mu = -5.0, sigma = 2.0)
                    ))

fit_tox <- bdeb_fit(mod_tox, chains = 4,
                  adapt_delta = 0.95, seed = 77)

```

```

# Diagnostics
bdeb_diagnose(fit_tox)

# EC50 and NEC with 95% credible intervals
bdeb_ec50(fit_tox, prob = 0.95)
# Output:
#   parameter mean median sd lower upper
#   EC50      ...    ...   ...   ...   ...
#   NEC       ...    ...   ...   ...   ...

# Dose-response plot with posterior uncertainty
plot_dose_response(fit_tox, n_draws = 200)

# Parameter summary
bdeb_summary(fit_tox,
  pars = c("p_Am", "p_M", "kappa", "k_d", "z_w", "b_w"))

```

8 Stan Model Details

This section describes the internal structure of the bundled Stan models for users who wish to understand or extend them.

8.1 DEB ODE Right-Hand Side

All models share the same ODE function implementing Equations (1)–(5). The function signature is:

```

vector deb_growth_ode(real t, vector x,
  array[] real theta, array[] real env)

```

where $\mathbf{x} = [E, V]'$, $\mathbf{theta} = \{p_{Am}, p_M, \kappa, v, E_G\}$, and $\mathbf{env} = \{f\}$.

A stabilisation term $1e-12$ is added to the denominator of the mobilisation flux to prevent division by zero:

$$\dot{p}^C = \frac{E v L}{E + [E_G] V + 10^{-12}}. \quad (10)$$

8.2 Prior Hyperparameters as Data

Prior hyperparameters are passed from R as Stan data variables (e.g., `prior_p_Am_mu`, `prior_p_Am_sd`). This allows users to change priors without recompiling the Stan model.

8.3 Hierarchical Non-Centred Parameterisation

The hierarchical model uses the standard non-centred parameterisation to avoid funnel geometries:

$$z_j \sim \mathcal{N}(0, 1), \quad j = 1, \dots, N, \quad (11)$$

$$\log\{p_{Am}\}_j = \mu_{\log p_{Am}} + \sigma_{\log p_{Am}} \cdot z_j. \quad (12)$$

This separates the population-level parameters (μ, σ) from individual-level deviations, greatly improving HMC sampling efficiency.

8.4 DEBtox: Damage Dynamics

The 4-state DEBtox model adds a scaled internal damage variable D_w :

$$\frac{dD_w}{dt} = k_d(\max(C_w - z_w, 0) - D_w). \quad (13)$$

At steady state, $D_w = \max(C_w - z_w, 0)$, so the stress factor is $s = b_w \cdot D_w$. The EC_{50} is computed analytically in `generated quantities` as $z_w + 0.5/b_w$.

9 Practical Advice

9.1 Choosing `adapt_delta`

Start with `adapt_delta = 0.8` (default). If `bdeb_diagnose()` reports divergent transitions, increase to 0.9, 0.95, or even 0.99. Higher values slow sampling but improve geometric fidelity.

9.2 When to Use Hierarchical Models

Use `type = "hierarchical"` when:

- More than 3–4 individuals are available.
- You want to estimate population-level distributions.
- Some individuals have sparse data (partial pooling helps).
- You need predictions for new, unobserved individuals.

9.3 Prior Sensitivity

Always compare results under at least two plausible prior specifications. Parameters that remain prior-dominated (posterior $\approx$ prior) are weakly identified and should be reported as such.

9.4 Computational Tips

- Stan model compilation is slow the first time (~ 30 s) but cached for subsequent fits.
- Use `parallel_chains = 4` to run chains in parallel.
- For large hierarchical models, start with a subset of individuals to debug parameterisation, then scale up.
- If ODE solver fails, check initial conditions (E0, L0) and ensure they are biologically reasonable.

10 Comparison with Existing Tools

Table 6: Comparison of **BayesianDEB** with existing DEB calibration tools.

Feature	AmP/DEBtool	deBInfer	Custom Stan	BayesianDEB
Bayesian inference	–	Yes	Yes	Yes
Pre-built DEB models	Yes	–	–	Yes
Hierarchical models	–	–	Manual	Built-in
DEBtox (TKTD)	Partial	–	Manual	Built-in
Prior specification API	–	Basic	Manual	Rich
Diagnostics	Minimal	Basic	Manual	Automated
PPC	–	–	Manual	Built-in
R package	MATLAB+R	Yes	–	Yes

Acknowledgements

This package is based on the theoretical framework developed in the *Bayesian DEB (BDEB)* monograph. The Stan Development Team provided the computational infrastructure that makes modern Bayesian DEB modelling practical.

References

- Carpenter, B., Gelman, A., Hoffman, M.D., et al. (2017). Stan: A probabilistic programming language. *Journal of Statistical Software*, 76(1), 1–32.
- Gelman, A., Vehtari, A., Simpson, D., et al. (2020). Bayesian workflow. *arXiv preprint arXiv:2011.01808*.
- Kooijman, S.A.L.M. (2010). *Dynamic Energy Budget Theory for Metabolic Organisation*. Cambridge University Press, 3rd edition.